

Lahore University of Management Sciences
School of Science and Engineering, Department of Artificial Intelligence

AI651 – Deep Learning for Space, Time and Graphs

Assignment 2 Report

Question 1: River Forecasting Proposal
Question 2: Motion or Memory?

Usama Azam
Roll number 25280109

GitHub repository:
`https://github.com/usama-azam/AI651-PA2`

Fall 2026 • Submitted October 2026

Contents

1 Question 1 – River Forecasting Proposal **2**

1.1 Architecture diagram . . . 2

1.2 Assumptions . . . 2

1.3 Forecasting task and data specification . . . 4

1.4 Model computations and design justification . . . 5

1.5 Operation and exceptional conditions . . . 7

1.6 Uncertainty, limitations, and additional information . . . 8

2 Question 2 – Motion or Memory? **10**

2.1 Setup . . . 10

2.2 Baseline: ConvLSTM (model A) . . . 11

2.3 Testing the baseline . . . 14

2.4 What can data fix? . . . 19

2.5 The final model . . . 21

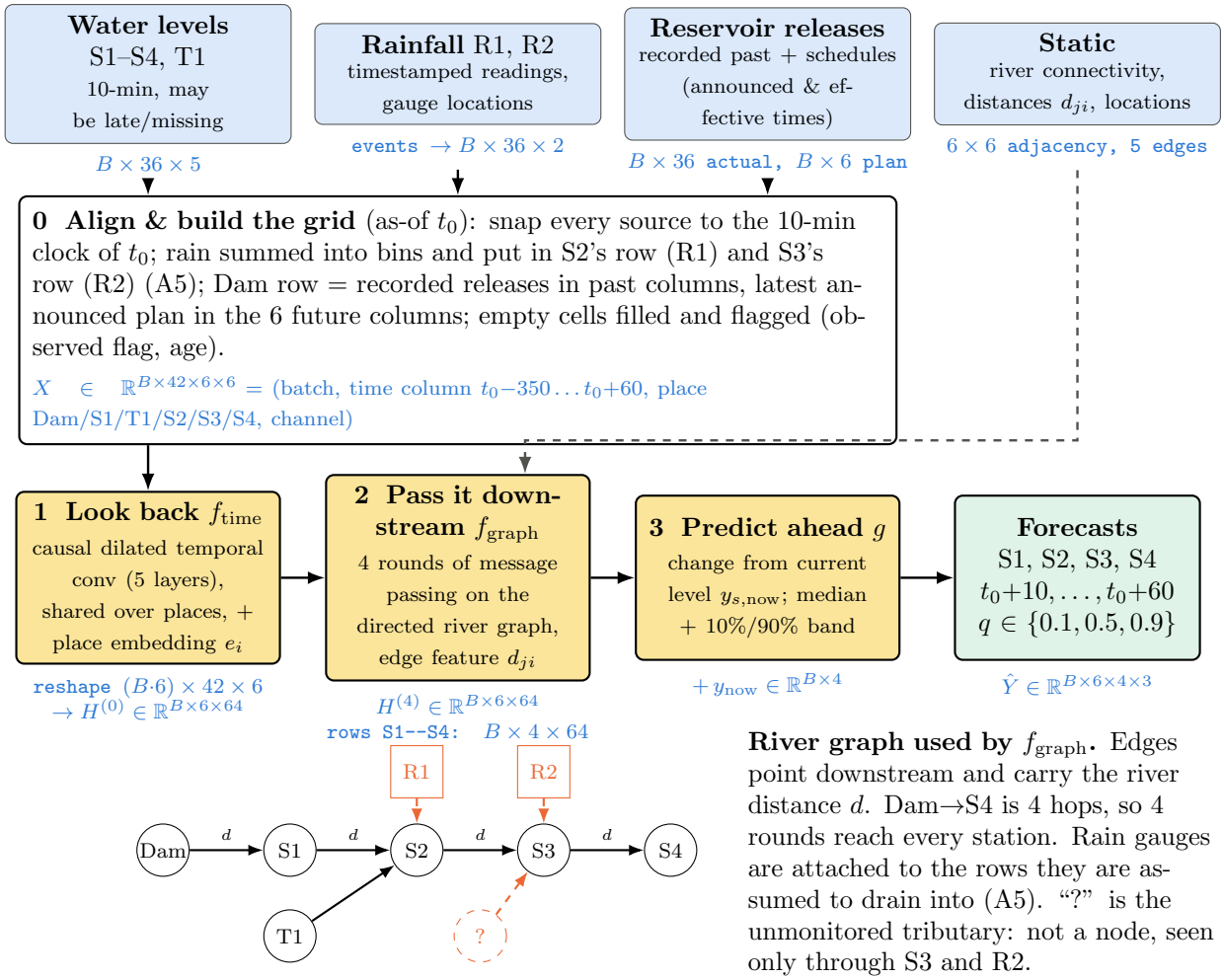
Appendix: Use of generative AI **27**

1 Question 1 – River Forecasting Proposal

Service. At every issuance time t_0 (every ten minutes) the system forecasts the water level at the four main-river stations S1–S4 for each ten-minute step of the next hour, $t_0 + 10, \ldots, t_0 + 60$ min. The one-hour-ahead value requested by the authority is the last of these six steps; forecasting the whole hour also shows how the level gets there. Each forecast is a median with a 10–90% band.

Design in one sentence. All sources are aligned on one grid (one row per place, one column per ten minutes); a temporal block summarises each place's recent history ("look back"); a directed graph block passes these summaries downstream along the river ("pass it downstream"); and a head predicts the change from the current level at S1–S4 ("predict ahead").

1.1 Architecture diagram



River graph used by f_{graph}. Edges point downstream and carry the river distance d. Dam→S4 is 4 hops, so 4 rounds reach every station. Rain gauges are attached to the rows they are assumed to drain into (A5). "?" is the unmonitored tributary: not a node, seen only through S3 and R2.

Figure Q1. Architecture. Top: data sources and their raw sizes. Block 0 aligns everything on one grid of 42 ten-minute columns (36 past columns ending at t_0, plus 6 future columns that only hold the announced reservoir plan) × 6 places × 6 channels (level, observed flag, rain, release, plan flag, age). Blocks 1–3 follow the course tutorial's skeleton (look back, pass downstream, predict ahead). Dashed grey: the static graph used by block 2. The head also receives the current levels y_{now}. Main tensor sizes in blue; B is the batch of issuance times.

1.2 Assumptions

Supplied facts (from the handout), kept separate from assumptions. F1: levels at S1–S4 and T1 are recorded every 10 minutes. F2: rainfall at R1 and R2 comes with timestamps and gauge

locations. F3: reservoir releases are recorded, and future release schedules are announced before each forecast, with announcement and effective times. F4: station locations, river connectivity and along-river distances are known; stations are unevenly spaced. F5: T1 measures a tributary joining between S1 and S2; an unmonitored tributary joins between S2 and S3. F6: travel times change with river conditions and are not supplied. F7: future rainfall and future levels are not available at issuance.

Assumptions (each: why it is reasonable, and what it changes in the design).

	I assume...	**because...**	**so in my design...**
A1	At least 5 years of history exist for all sources, including several floods.	Authorities usually keep long gauge records; floods are needed to learn high-water behaviour.	Train/validation/test split *by time* (e.g. last year = test); high-water weighting in the loss (Sec. 4).
A2	Levels are in metres above a fixed gauge datum per station, consistent over time.	Standard practice for river gauges.	The model predicts the *change* from the current level; each station is normalised by its own mean and spread.
A3	All timestamps use one clock (UTC) and give the time the value refers to; level readings fall within ±5 min of the 10-min grid.	Telemetry systems log with a central clock; small offsets are typical.	Values are snapped to the nearest grid column (Sec. 3).
A4	Each rain reading is the total since the previous reading of that gauge, and gauges may report irregularly.	The handout gives timestamps but not what a reading means; tipping-bucket gauges report totals.	Rain is summed into 10-min bins $(t-10, t]$; a gap is treated as missing, not as zero rain.
A5	R1 drains into the reach between S1 and S2 (including T1's catchment); R2 drains into the unmonitored tributary.	R1 sits on the hills above the S1–S2 reach; R2 sits above the unmonitored stream (handout map).	R1's rain goes into S2's row, R2's into S3's row of the grid. R2 is the main clue for the unmonitored inflow.
A6	In the slowest case, water takes at most about 4 h from the dam to S4, and rain reaches the river within about 4 h.	A small mountain river with four stations; travel times fall at high flow (F6).	History window of 6 h (36 steps): every source that can affect any of the six target times lies inside it, with margin.
A7	Water only flows downstream; no backwater or tidal effects reach upstream stations.	Steep river below a dam (handout picture).	Graph edges point downstream only; a station's forecast cannot use information from stations below it.
A8	Travel time and attenuation depend on the flow state, which is visible in the current levels.	Deeper water flows faster; this is the usual hydraulic behaviour.	The delay is *learned* from the levels, not fixed; the look-back covers the slowest case (A6).
A9	Tributary and reservoir inflows add up at the junctions.	Conservation of water.	Messages from several upstream neighbours are *summed* at a junction (S2 receives S1 and T1).

	I assume...	because...	so in my design...
A10	Gaps are mostly short (minutes to hours); long outages are rare.	Typical telemetry behaviour.	Missing values are filled and flagged, and the model is trained with randomly hidden inputs (Sec. 5).
A11	Gauge data normally arrive within 30 min; delays up to 1 h occur.	Radio/cellular telemetry with buffering.	An "age" channel tells the model how old each value is; training simulates these delays.
A12	A release schedule is a list of (effective time, flow) steps; without a newer plan, the last planned or actual release continues.	Common way of announcing reservoir operations.	Future columns of the dam row hold the latest plan announced at or before t_0, with a plan flag.
A13	Each station has a known high-water (warning) level; if not supplied, its 95th percentile is used.	Authorities define warning levels; the percentile is a fallback.	Errors above this level get a larger weight in the loss.
A14	River geometry and gauge rating do not change much over the record (no major works).	Needed for one model trained on the past to work now.	One shared model; periodic retraining and monitoring for drift (Sec. 6).

1.3 Forecasting task and data specification

Issuance time. t_0 is every ten minutes on the hour grid (09:00, 09:10, ...). A forecast issued at t_0 may only use information that has *arrived* by t_0 ("if you would not know it at t_0, the model cannot use it"). A plan announced at 09:10 is therefore not used by the 09:00 forecast.

History window. 6 hours: the 36 grid columns $t_0 - 350, \ldots, t_0$ min (A6). **Horizon.** One hour, as six steps $t_0 + 10k$ min, $k = 1, \ldots, 6$. The grid also has 6 *future* columns $t_0 + 10, \ldots, t_0 + 60$; only the dam row is filled there (announced plan), all other future cells are empty and flagged.

Outputs. For each station $s \in \{\mathrm{S1}, \ldots, \mathrm{S4}\}$, step k and quantile $q \in \{0.1, 0.5, 0.9\}$: $\hat{y}_{s,k}^{(q)}$, the forecast level at $t_0 + 10k$. The authority's one-hour forecast is $k = 6$; the median is the point forecast and the 0.1–0.9 band is an 80% range.

Item	Dimensions	Axes and meaning	Time coverage (rel. t_0)
Grid X (model input)	$B \times 42 \times 6 \times 6$	batch of issuance times; time column $\tau = 1..42$; place (Dam, S1, T1, S2, S3, S4); channel (below)	$\tau = 1..36$: $t_0{-}350..t_0$; $\tau = 37..42$: $t_0{+}10..t_0{+}60$
ch. 1 level	(in X)	level of S1–S4, T1 (normalised); 0 in Dam row	past columns only
ch. 2 observed	(in X)	1 if a measured value for that cell had arrived by t_0, else 0	past columns; 0 in future
ch. 3 rain	(in X)	10-min rain total of R1 (S2 row) and R2 (S3 row); 0 elsewhere	past only (future rain unknown)
ch. 4 release	(in X)	reservoir release flow, Dam row only	past: recorded; future: plan
ch. 5 plan flag	(in X)	1 where ch. 4 is an announced plan (not a record)	future columns of Dam row
ch. 6 age	(in X)	hours since the value in ch. 1/4 was measured (0 if fresh; > 0 if filled)	past columns
Current level y_{now}	$B \times 4$	latest received level at S1–S4	$\leq t_0$
Static graph	6×6, 5 edges	directed adjacency (Dam→S1, S1→S2, T1→S2, S2→S3, S3→S4), distance d_{ji} per edge	fixed
Output $\hat{Y}$	$B \times 6 \times 4 \times 3$	batch; step k; station S1–S4; quantile 0.1/0.5/0.9	$t_0{+}10..t_0{+}60$
Target Y, mask M	$B \times 6 \times 4$ each	true future levels; $M = 1$ if the target was measured (training only)	$t_0{+}10..t_0{+}60$

Table Q1. Inputs and outputs of one forecast (B = number of issuance times in a batch; $B = 1$ in operation).

Alignment across sampling times. All sources are put on the 10-minute clock of t_0. Levels are snapped to the nearest column (A3); rain readings are summed into bins $(t-10, t]$ (A4); releases are averaged over each bin; plans are converted into a step function and sampled at the future columns (A12). Nothing measured after t_0 enters X, and in training each sample is rebuilt *as it was* at t_0 (the data that had arrived by then, and the plan that was valid then), so training matches operation.

Alignment across locations. Each place gets its own row, so no spatial interpolation is needed. Rain gauges are not stations; they are attached to the row of the reach they are assumed to drain into (A5). Physical distances, which are uneven (F4), are not encoded by the row order but enter as edge features d_{ji} of the graph.

Available at issuance: past levels (S1–S4, T1) and rain (R1, R2) that have arrived by t_0, recorded releases up to t_0, the latest release plan announced at or before t_0, and the static river description. **Not available:** future rain, future levels, the unmonitored tributary's inflow, exact travel times.

1.4 Model computations and design justification

Notation. Places $i \in \mathcal{V} = \{\text{Dam}, \text{S1}, \text{T1}, \text{S2}, \text{S3}, \text{S4}\}$; stations $s \in \mathcal{S} = \{\text{S1}, \ldots, \text{S4}\}$; columns $\tau = 1, \ldots, 42$; $x_{i,\tau} \in \mathbb{R}^6$ is one grid cell; $d = 64$ is the hidden width; $\mathcal{U}(i)$ is the set of places directly upstream of i; d_{ji} is the river distance from j to i divided by the mean edge length.

Block 1 – look back (f_{time}, one per place, weights shared across places). A causal dilated temporal

convolution (the course's T block with growing dilation):

$$z_{i,\tau}^{(0)} = W_{\text{in}}\, x_{i,\tau} + e_i + p_\tau, \tag{1}$$

$$z_{i,\tau}^{(\ell)} = z_{i,\tau}^{(\ell-1)} + W_o^{(\ell)}\,\text{ReLU}\Big(\sum_{m=0}^{2} W_m^{(\ell)}\, z_{i,\tau-m\delta_\ell}^{(\ell-1)} + b^{(\ell)}\Big), \qquad \delta_\ell = 2^{\ell-1},\ \ell = 1,\dots,5, \tag{2}$$

$$h_i^{(0)} = \text{LayerNorm}\big(z_{i,42}^{(5)}\big) \in \mathbb{R}^d. \tag{3}$$

e_i is a learned embedding of place i (lets a shared block behave differently at each place); p_τ is a learned embedding of the column (past vs. future, how long ago); terms with $\tau - m\delta_\ell < 1$ are zero. The temporal reach is $1 + 2(1+2+4+8+16) = 63 \geq 42$ columns, so $h_i^{(0)}$ summarises the whole 6-h history and the planned hour.

Block 2 – pass it downstream (f_{graph}, $r = 1,\dots,4$ rounds on the directed river graph):

$$m_{j\to i}^{(r)} = \phi^{(r)}\big(\big[\, h_j^{(r-1)}\,;\, h_i^{(r-1)}\,;\, d_{ji}\,\big]\big), \qquad j \in \mathcal{U}(i), \tag{4}$$

$$h_i^{(r)} = h_i^{(r-1)} + \psi^{(r)}\Big(\Big[\, h_i^{(r-1)}\,;\, \textstyle\sum_{j\in\mathcal{U}(i)} m_{j\to i}^{(r)}\,\Big]\Big), \tag{5}$$

with $\phi^{(r)}, \psi^{(r)}$ two-layer MLPs (ReLU) and the sum empty for the Dam and T1. After 4 rounds every station has a path from every place upstream of it (Dam→S4 is 4 hops).

Block 3 – predict ahead. A linear head $G : \mathbb{R}^d \to \mathbb{R}^{6\times 3}$ gives, for each station s and step k, three numbers $(a_{s,k}, b_{s,k}, c_{s,k}) = G(h_s^{(4)})_k$; with σ_s the station's standard deviation,

$$\hat{y}_{s,k}^{(0.5)} = y_{s,\text{now}} + \sigma_s\, a_{s,k}, \qquad \hat{y}_{s,k}^{(0.1)} = \hat{y}_{s,k}^{(0.5)} - \sigma_s\,\text{softplus}(b_{s,k}), \qquad \hat{y}_{s,k}^{(0.9)} = \hat{y}_{s,k}^{(0.5)} + \sigma_s\,\text{softplus}(c_{s,k}). \tag{6}$$

Training objective. Weighted pinball (quantile) loss over observed targets, with $\rho_q(u) = \max(qu, (q-1)u)$:

$$\mathcal{L} = \frac{\sum_{b,s,k} w_{b,s,k}\, M_{b,s,k} \sum_{q\in\{0.1,0.5,0.9\}} \rho_q\big((y_{b,s,k} - \hat{y}_{b,s,k}^{(q)})/\sigma_s\big)}{\sum_{b,s,k} w_{b,s,k}\, M_{b,s,k}}, \qquad w_{b,s,k} = 1 + 4\cdot \mathbb{1}\big[y_{b,s,k} > \theta_s\big], \tag{7}$$

where θ_s is the high-water level (A13). Training uses Adam with early stopping on a later time period (A1); inputs are randomly hidden and delayed during training (Sec. 5).

Justification of the main choices.

- *Information access first.* A station's own history cannot tell it that a wave is on its way; the information is upstream. A purely temporal model (T only) would be blind to it; the graph block creates the missing spatial path, and the look-back creates the temporal path. Spatial reach (4 hops) and temporal reach (63 columns > 42) cover every upstream source within the window (A6).
- *T first, then graph.* Each place has a long, regular time series, while the graph is tiny (6 nodes, 5 edges). Summarising time first is cheap, and h_j keeps *when* things happened at j (its receptive field covers the whole window), so the next block can still work out arrival times.
- *Directed graph with distances, not a chain of rows or full attention.* Adjacent rows are not equally far apart (F4), so distance is an explicit edge feature. Edges point downstream (A7), which builds in known physics and saves data, important because floods are rare (A1). Joint attention over all $42 \times 6 = 252$ tokens would also be affordable, but it would have to learn the flow direction from data; I keep it as an alternative if validation shows the graph is too restrictive.
- *Learned delays.* Travel times change (F6, A8): calm ≈ slow, flood ≈ fast. The model is not given a fixed lag; it learns the delay from the levels themselves, and the 6-h window covers the slowest case.

- *Change from now.* At one hour, persistence ($\hat{y} = y_{\text{now}}$) is already a strong forecast; predicting the change makes the model learn the correction and keeps outputs in a realistic range.
- *Quantiles and weights.* The median minimises absolute error (more robust to outliers than the mean), and the 10/90% band tells the authority how sure the forecast is, which matters more than a single number for flood decisions. The weight w makes errors at high water count five times more, because those are the forecasts that matter and they are rare. The softplus widths guarantee $\hat{y}^{(0.1)} \le \hat{y}^{(0.5)} \le \hat{y}^{(0.9)}$.

How each source influences each forecast (paths in Figure Q1).

- *Reservoir releases*: the Dam row reaches S1 in 1 round and S4 in 4 rounds. Past releases explain water already on its way; the announced plan in the future columns tells S1 (and, for slow water, S2) what will be released during the forecast hour.
- *Upstream conditions*: S1's summary reaches S2, S2's reaches S3, and so on; each station's forecast is driven mainly by its upstream neighbour's recent history, shifted by a learned delay.
- *Tributaries*: T1 enters S2 as a second message (summed, A9). The unmonitored tributary has no node; its effect is learned from R2's rain in S3's row and from how S3 behaves compared with S2.
- *Rainfall*: R1 enters at S2, R2 at S3, through those rows' look-back; only rain that has already fallen is used, so its effect is the delayed runoff that is already on its way.

Physical assumptions vs. learned mappings. Built in from physics or the data: flow direction and connectivity (edges, A7), distances (edge features), addition of inflows at junctions (summed messages, A9), causality (no future rain or levels), and which reach each gauge drains into (A5). Learned from data: travel times and how they change with flow, attenuation of waves, rainfall-to-runoff response, the size of the unmonitored inflow, and the forecast uncertainty.

Cost. About 1.9×10^5 parameters; one forecast needs about 2×10^7 multiply–adds (5 temporal layers over 6×42 cells at width 64; the 4 graph rounds on 5 edges are negligible).

1.5 Operation and exceptional conditions

(i) Missing observations. A missing level is filled with the last value received before it (never with a later one), its *observed* flag is set to 0 and its *age* channel records how old the filled value is. Missing rain is filled with 0 and flagged as unobserved (not "no rain"). If $y_{s,\text{now}}$ itself is missing, the last received value is used and its age is passed on. Because the model is trained with randomly hidden cells and whole-station outages, it learns to rely more on upstream stations when a station is silent; the quantile band widens because such samples had larger errors in training. If a whole station is down for more than the window, the forecast is still produced from its neighbours and marked as degraded in the output.

(ii) Delayed observations. The system never waits: at t_0 it uses what has arrived (as-of data). A value that arrives late is treated as missing at t_0 (with its age), and the next run at $t_0 + 10$ uses it, rebuilding the past columns of the grid. Training simulates the delays of A11, so the model has seen "stale" inputs before.

(iii) Announced future releases. The latest schedule announced at or before t_0 fills the dam row's future columns (plan flag = 1). A schedule announced after t_0 is not used for this forecast. If no schedule covers the hour, the last release is continued (A12) with the plan flag = 0, so the model knows it is a guess.

(iv) Scheduled vs. actual releases. Past columns always contain the *recorded* release, never the plan; future columns contain the plan. In training, future columns are filled with the plan *as announced at that time*, not with the release that later happened. The model therefore learns how far plans can be trusted, and the uncertainty band includes typical deviations. When the dam deviates,

the deviation appears in the recorded past release at the next issuance (10 minutes later) and the forecast updates; if deviations become systematic, the plan is down-weighted by retraining.

Feasibility of running every ten minutes. One forecast is a single forward pass of a $\sim 2 \times 10^5$-parameter network on a $42 \times 6 \times 6$ grid, well under a second on an ordinary CPU; data collection and grid building are simple table operations. Training is done offline (hours on one GPU) and repeated, for example, monthly. The 10-minute cycle is therefore limited by data arrival, not by computation.

1.6 Uncertainty, limitations, and additional information

Uncertainty / limitation	**Forecast impact**	**Response, or what remains unresolved**
Unmonitored tributary	Unknown inflow between S2 and S3 makes S3 (and S4) under-predicted during local storms.	R2 (A5) and the S2-to-S3 difference act as proxies; the band is wider at S3–S4. Unresolved without a gauge (request 1).
Changing travel times	A fixed lag would be wrong in floods (faster) or calm periods (slower).	Delay is learned from levels (A8); 6-h window covers the slow case. Floods larger than any in the record remain a risk (extrapolation).
Future rainfall unknown	Heavy rain that starts after t_0 cannot be anticipated; rises from local runoff can be missed, mostly at the end of the hour.	Only already-fallen rain is used; the band reflects this. Rain radar/nowcasts would help (request 3).
Only two rain gauges	Point readings miss storms between gauges.	Rain mapping is an assumption (A5); errors show up as wider bands at S2/S3.
Rare floods	Few training examples of the most important situations.	High-water weight in the loss; time-based validation that includes a flood; report errors separately above θ_s.
Release plan deviations	Forecast for S1 (and later S2) is off when the dam does not follow the plan.	Corrected at the next issuance from the recorded release (Sec. 5 iv); trained on plans as announced.
Sensor problems and drift (debris, datum changes, rating changes)	Wrong inputs propagate downstream through the graph.	Range and jump checks before the grid is built; flagged values are treated as missing; periodic retraining (A14).
Calibration of the band	An 80% band may not contain 80% of outcomes.	Check coverage on the test period; widen or narrow with a simple calibration step if needed.

	Additional information requested	**How it would help**	**Usable without it?**
1	A level gauge (or flow estimate) on the unmonitored tributary; at least its catchment area.	Adds the missing inflow between S2 and S3 as a node; largest expected gain at S3/S4.	Yes, with R2 as a proxy and wider bands.

	Additional information requested	**How it would help**	**Usable without it?**
2	Record length, reporting delays and data-quality notes per source.	Confirms or replaces A1, A3, A10, A11; sets the delay simulation in training.	Yes, with the stated assumptions.
3	Rain radar or rainfall nowcasts available at t_0.	Shows rain between gauges and rain about to fall; less under-prediction at the end of the hour.	Yes.
4	Official warning levels, rating curves and cross-sections per station.	Sets θ_s for the loss and the warnings; allows checking floods larger than in the record.	Yes, using percentiles (A13).
5	Reservoir operating rules and the history of plan vs. actual releases.	Better treatment of plan deviations and of releases when no plan is announced.	Yes.
6	Reservoir level and spillway status.	Lets the Dam row carry a level as well as a release; helps in spill situations.	Yes.
7	Past travel-time studies or recorded flood waves between stations.	Checks the learned delays and the 6-h window (A6).	Yes.

2 Question 2 – Motion or Memory?

Implementation summary. All code is in the repository (notebooks Step 1–6). Moving MNIST is loaded with `torchvision.datasets.MovingMNIST(split=None)`; videos 0–7999 are used for training, 8000–8999 for validation and 9000–9999 for testing (test videos are numbered 0–999 below). All models were trained and evaluated on Kaggle with an **NVIDIA Tesla T4** GPU (PyTorch 2.11), with Adam (learning rate 10^{-3}, no schedule), batch size 32, 30 epochs, final-epoch weights, seeds 0 and 1. Table entries are mean (seed 0, seed 1); errors use three significant figures and scientific notation below 10^{-3}.

2.1 Setup

	Check	Expected	Mine
(a)	$\max \lvert M(M(x)) - x \rvert$	0	0 (`torch.equal` = True)
(b)	$\max \lvert R^4(x) - x \rvert$	0	0 (`torch.equal` = True)
(c)	$\max \lvert MRM(x) - R^{-1}(x) \rvert$	0	0 (`torch.equal` = True)
(d)	bright pixel of $M(p)$ (frame, row, column)	(7, 10, 43)	(7, 10, 43)
(e)	bright pixel of $R(p)$ (frame, row, column)	(7, 43, 10)	(7, 43, 10)
(f)	1× input / target frames	1–5 / 6–10	1–5 / 6–10
(g)	2× input / target frames	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19
(h)	3× input / target frames	1, 4, 7, 10, 13 / 16, 19	1, 4, 7, 10, 13 / 16, 19

Table 1. Checks of the transformation and window functions (all eight pass).

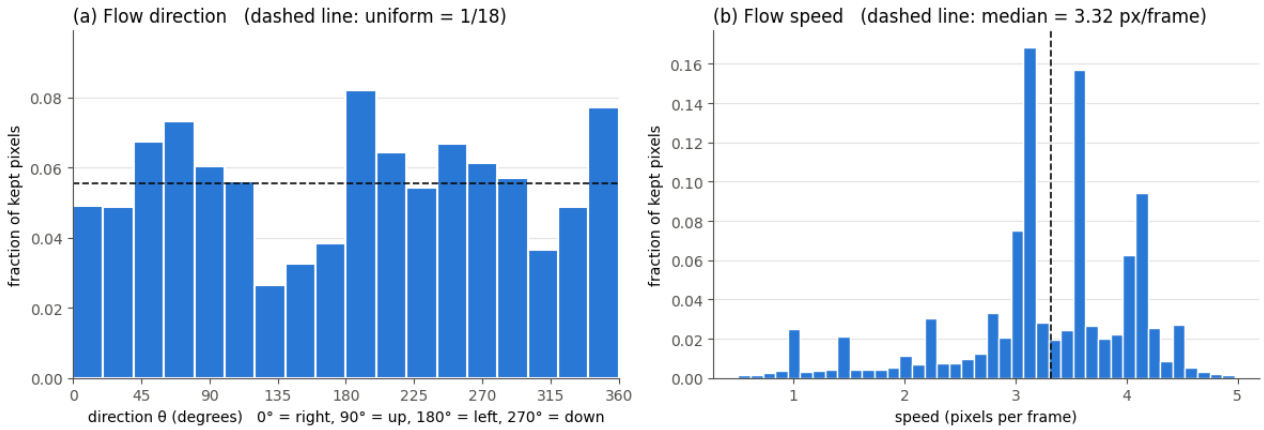


Figure 1. Farneback optical flow on the first 50 training videos (950 consecutive frame pairs). (a) Flow direction $\theta = \text{atan2}(-d_y, d_x)$, 18 bins of 20°, dashed line at 1/18. (b) Flow speed $\sqrt{d_x^2 + d_y^2}$ in pixels per frame; dashed line at the median, **3.32 px/frame**. Both as fractions of the **103,050 kept pixels**.

Q2.1 Directions. Yes, as far as the direction of motion is concerned. Figure 1(a) shows that digits in the training data move in every direction: every bin is above zero, the lowest being 0.026 (120–140°) and the highest 0.082 (180–200°), compared with $1/18 \approx 0.056$ for a uniform distribution. A mirror changes a direction θ into $180° - \theta$, and a 90° rotation changes it into $\theta + 90°$; for example, a digit moving right (0°) moves left (180°) after a mirror and upward (90°) after a rotation. If every training video were mirrored, the bar at θ would move to $180° - \theta$; if every video were rotated, every bar would shift by 90°. In both cases every bin would still be well above zero, so a mirrored or rotated video contains no direction that is new to a model trained on this data. However, the directions are not equally common, so the transformed histogram would not be identical to the

original: for example, after a mirror the 120–140° bin would receive the pixels of the 40–60° bin, and its fraction would rise from 0.026 to 0.067. Some directions would therefore be more common at test time than in training, but none would be unseen. This only concerns motion; mirrored and rotated digits also look different, which is discussed in Q2.2.

Speed. The typical speed is the median, 3.32 px/frame. Playing a video at 2× means keeping every second frame ($k = 2$), so each digit moves twice as far between the frames the model sees: $2 \times 3.32 = 6.64$ px/frame. At 3× ($k = 3$) it moves $3 \times 3.32 = 9.96$ px/frame. Figure 1(b) contains no kept pixels faster than about 5 px/frame, so these speeds never occur in training: unlike mirrored and rotated motion, faster motion is unfamiliar to a model trained on this data.

Overlaps. No. Optical flow can be wrong where digits overlap, so some pixels get a wrong direction or speed. But overlaps happen only in the few frames where digits cross, so these pixels are a small minority of the 103,050 kept pixels. Their errors spread over many bins: they can change bar heights slightly, but they cannot push a bar to zero or make one much too high, so they act as noise rather than a systematic bias. Even the rarest direction bin holds about 2,700 pixels ($0.026 \times 103{,}050$), most of them from frames without overlap. The median speed is also robust, because it hardly moves when a small fraction of values is wrong. So the conclusions above do not change.

Q2.2

Transformation	Motion statistics	Digit appearance
Mirror	Almost unchanged: each direction θ becomes $180° - \theta$, but every direction already occurs in training (Q2.1), and the speed is the same.	Changed: digits are reversed left to right (e.g. a mirrored 2 or 7). Nearly symmetric digits such as 0, 1 and 8 look almost the same.
Rotation by 90°	Almost unchanged: each direction θ becomes $\theta + 90°$. All directions are still familiar, and the speed is the same.	Strongly changed: vertical strokes become horizontal and digits become wider than tall (e.g. a 1 becomes a horizontal bar), so almost every digit looks unlike the training digits.
Faster (2×, 3×)	Changed: directions stay the same, but the speed rises from 3.32 to 6.64 and 9.96 px/frame, which never occurs in training (Q2.1).	Unchanged: every frame is an ordinary frame of the original video.

Mirror and rotation mainly change the appearance of the digits and keep the motion familiar, whereas faster playback changes only the motion and keeps the appearance familiar. Each transformation therefore changes one factor and holds the other fixed, as in a controlled experiment that changes one variable at a time. Everything else is also held fixed: the same model is tested on the same test videos; mirror and rotation only rearrange pixels (the Table 1 checks give exactly 0); faster playback only skips frames; and the motion rule (straight lines and bounces off the walls) remains valid under all three, so the correct future frames $T(Y)$ are always known and a model that has really learned the motion rule should cope with all three. If a model fails under faster playback, this points to unfamiliar motion, because the digits look normal. If it fails under mirror or rotation, this points to unfamiliar appearance, because the directions and speeds are already familiar. Comparing mirror with rotation adds a further check: both keep the motion familiar, but rotation changes the appearance more, so a larger error under rotation would also point to appearance rather than motion.

2.2 Baseline: ConvLSTM (model A)

Model A follows the handout: `pixel_unshuffle`(4) to a 16×16 grid with 16 channels, two ConvLSTM layers (Shi et al., 2015, without peepholes) with 64 hidden channels and 3×3 kernels (each layer's four gates come from one convolution on $[u_\tau, h_{\tau-1}]$), a 1×1 convolution to 16 channels, `pixel_shuffle`(4)

and a sigmoid. It is unrolled for 9 steps without teacher forcing (steps 6–9 take its own previous prediction as input); the outputs of steps 5–9 form $\hat{Y}$. Parameters: 184,576 + 295,168 + 1,040 = **480,784**. Loss: squared error averaged over pixels and available target frames of each video, then over videos. Training time 11.0 (10.9, 11.2) s per epoch on a T4; final validation $E^{(5)}$ 0.0222 (seed 0) and 0.0218 (seed 1).

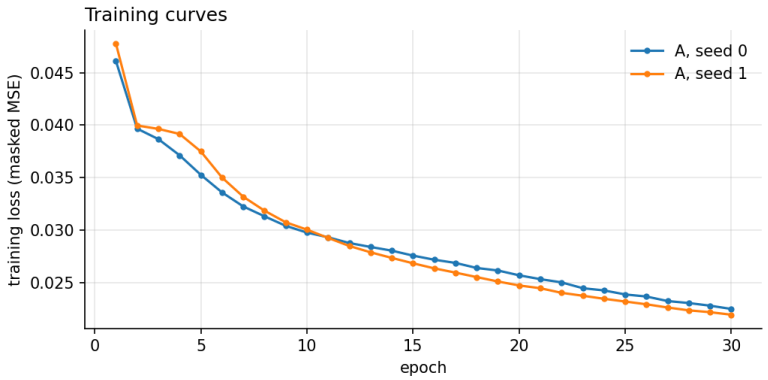


Figure S1. Training loss of model A (both seeds). (Supplementary; not a handout figure.)

Q2.3 Model A's reach is **8 original pixels per time step** along rows and columns (about 11.3 px diagonally). The reasoning:

1. **Patch size.** `pixel_unshuffle`(4) folds each 4×4 patch into one cell of a 16×16 grid, so neighbouring cell centres are 4 original pixels apart: moving one cell means moving 4 px. Following the conventions, the patch width itself is not added, and unshuffle, shuffle and the 1×1 output convolution move no information.
2. **Kernel size.** All four gates of a ConvLSTM layer come from one 3×3 convolution on the input and the previous hidden state. A 3×3 kernel lets each cell see only itself and its 8 neighbours, so one convolution moves information at most 1 cell. The other gate operations (σ, tanh, $\times$, $+$) act on each cell separately and move nothing. So each layer moves information at most 1 cell per step.
3. **Number of layers.** The two layers are stacked: the path from the new input u_τ passes through the layer-1 convolution and then the layer-2 convolution, so it moves 2 cells.
4. **Recurrence.** The previous states enter through the same convolutions. The path from $h^{(1)}_{\tau-1}$ passes through both convolutions (2 cells); the path from $h^{(2)}_{\tau-1}$ passes only through the layer-2 convolution (1 cell). The cell states c are updated cell by cell, so they add nothing. The recurrence lets information travel further over several steps, but each single step adds at most 2 cells.

The longest paths (from u_τ and from $h^{(1)}_{\tau-1}$) move 2 cells $= 2 \times 4 =$ **8 px** along a row or column. Because a 3×3 kernel also reaches diagonal neighbours, information can move 2 cells in both directions at once: $\sqrt{8^2 + 8^2} = 8\sqrt{2} \approx 11.3$ px. So in one update, the network can connect a digit's old position (held in its state) with its new position only if the digit moves at most about 8 px horizontally and 8 px vertically per step.

Model	$\mathbf{MSE}_1$	$\mathbf{MSE}_2$	$\mathbf{MSE}_3$	$\mathbf{MSE}_4$	$\mathbf{MSE}_5$	$E^{(5)}$	Params
A	0.0172 (0.0175, 0.0170)	0.0205 (0.0208, 0.0203)	0.0228 (0.0231, 0.0225)	0.0245 (0.0248, 0.0242)	0.0262 (0.0264, 0.0259)	0.0222 (0.0225, 0.0220)	480,784
Blank	0.0426	0.0426	0.0426	0.0425	0.0425	0.0426	0
Copy-last	0.0492	0.0604	0.0669	0.0708	0.0732	0.0641	0

Table 2. Test set, 1×, $t = 1$. Model A: mean (seed 0, seed 1).

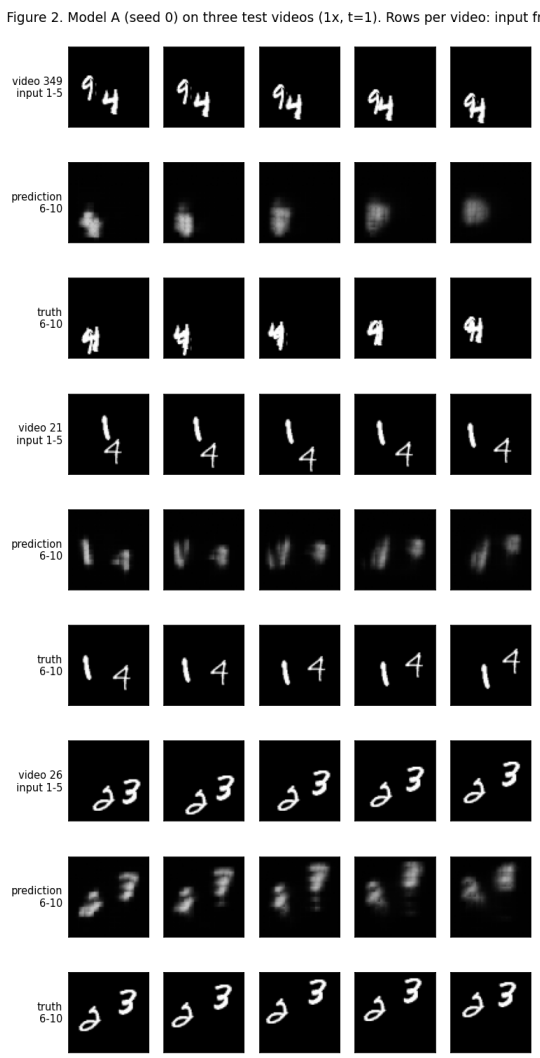


Figure 2. Model A (seed 0) on test videos 349 (digits overlapping), 21 and 26: input frames 1–5, prediction of frames 6–10, ground truth.

Q2.4 Model A clearly beats both reference predictors. Its $E^{(5)}$ is 0.0222 (0.0225, 0.0220), about 2× lower than Blank (0.0426) and about 3× lower than Copy-last (0.0641), and it is lower than both at every horizon (Table 2). The two seeds differ by only about 2%, so the result is reliable. Copy-last is worse than Blank even though it looks like a sensible guess, because of the double penalty: the digits move about 3.3 px per frame and their strokes are thin, so a copied digit lands mostly in the wrong place and is wrong twice, dark where the digit really is and bright where it no longer is. Blank is only wrong once, where the digit is.

Model A's error grows steadily with the horizon, from $\mathrm{MSE}_1 = 0.0172$ to $\mathrm{MSE}_5 = 0.0262$ (about +50%), while Blank stays flat at about 0.0426; even at $h = 5$ model A is still well below it. Figure 2 shows why: at $h = 1$ the digits are in the right places and still recognisable, but they become blurrier

and fainter at every step. From step 6 on the model feeds its own predictions back as input, so small errors build up, and it becomes less certain where each digit will be. Under squared error the best prediction under uncertainty is an average of the possible positions, which is a blur; a blur costs less than committing to one sharp guess that might be wrong, for the same double-penalty reason as Copy-last. In videos 21 and 26 the blobs still roughly follow the true positions, so the motion is tracked even though the digits' shapes are lost.

When digits overlap, the prediction breaks down further. In video 349 the 9 and the 4 overlap in the last input frames, and model A merges them into a single blurry blob: in the prediction one can no longer tell which digit is which, and the model does not follow each digit's own motion.

Q2.5 *(written before running any experiment in Testing the Baseline; reproduced unedited)*

By "pass" I mean that the error on transformed videos, E_T, stays close to the normal error E.

Mirror: I predict that model A will pass the mirror test. Figure 1(a) shows that every direction already occurs in the training data, so mirrored motion is not new to the model. A mirror also changes the digits only a little: it reverses them left to right, and many strokes still look similar.

Rotation: I predict that model A will also pass the rotation test. Again, every direction already occurs in training (Figure 1(a)), and a rotation does not change the speed. Since rotation, like the mirror, keeps the motion familiar, I expect the same result as for the mirror.

Speed: I predict that model A will work at 1× and break down at 2× and higher. At 1×, digits move about 3.32 px per step, the typical training speed (the median in Figure 1(b)). At 2× and 3× they move 6.64 and 9.96 px per step, and these speeds are rare in training (Figure 1(b)), so the model has hardly seen motion this fast. My reach estimate (Q2.3) is 8 px per step along rows and columns. At 2×, 6.64 px is still inside this reach, so I expect the breakdown at 2× to come from the unfamiliar speed rather than from the reach. At 3×, 9.96 px is also beyond the reach (except for nearly diagonal motion, which can reach up to about 11.3 px), so both reasons apply.

2.3 Testing the baseline

Model	$E^{(5)}$	C_M	E_M	C_R	E_R
A	0.0222 (0.0225, 0.0220)	0.00163 (0.00155, 0.00171)	0.0228 (0.0231, 0.0226)	0.00288 (0.00254, 0.00321)	0.0240 (0.0240, 0.0239)
Blank	0.0426	0	0.0426	0	0.0426
Copy-last	0.0641	0	0.0641	0	0.0641

Table 3. Consistency gaps and transformed errors (test set, 1×, $t = 1$). Model A: mean (seed 0, seed 1).

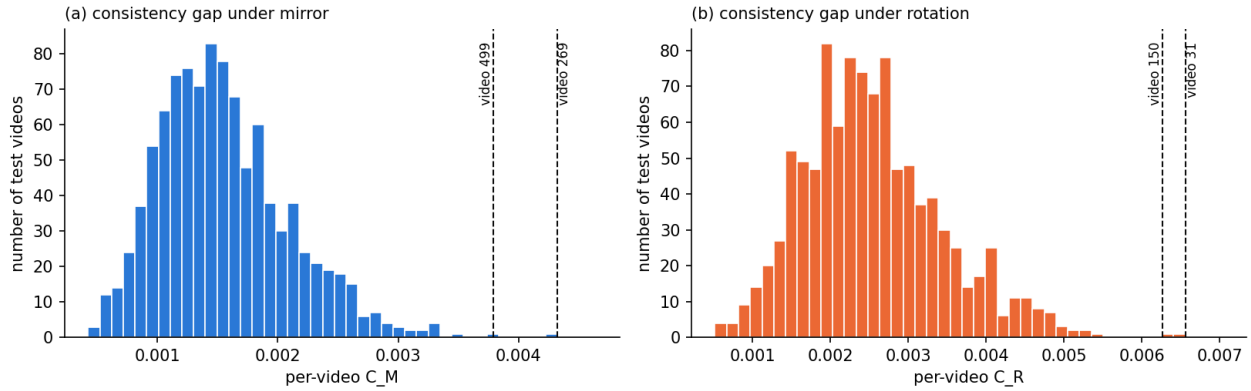


Figure 4. Histograms of the per-video C_M (a) and C_R (b) of the seed-0 model A over the 1,000 test videos; dashed lines mark the Figure 3 videos.

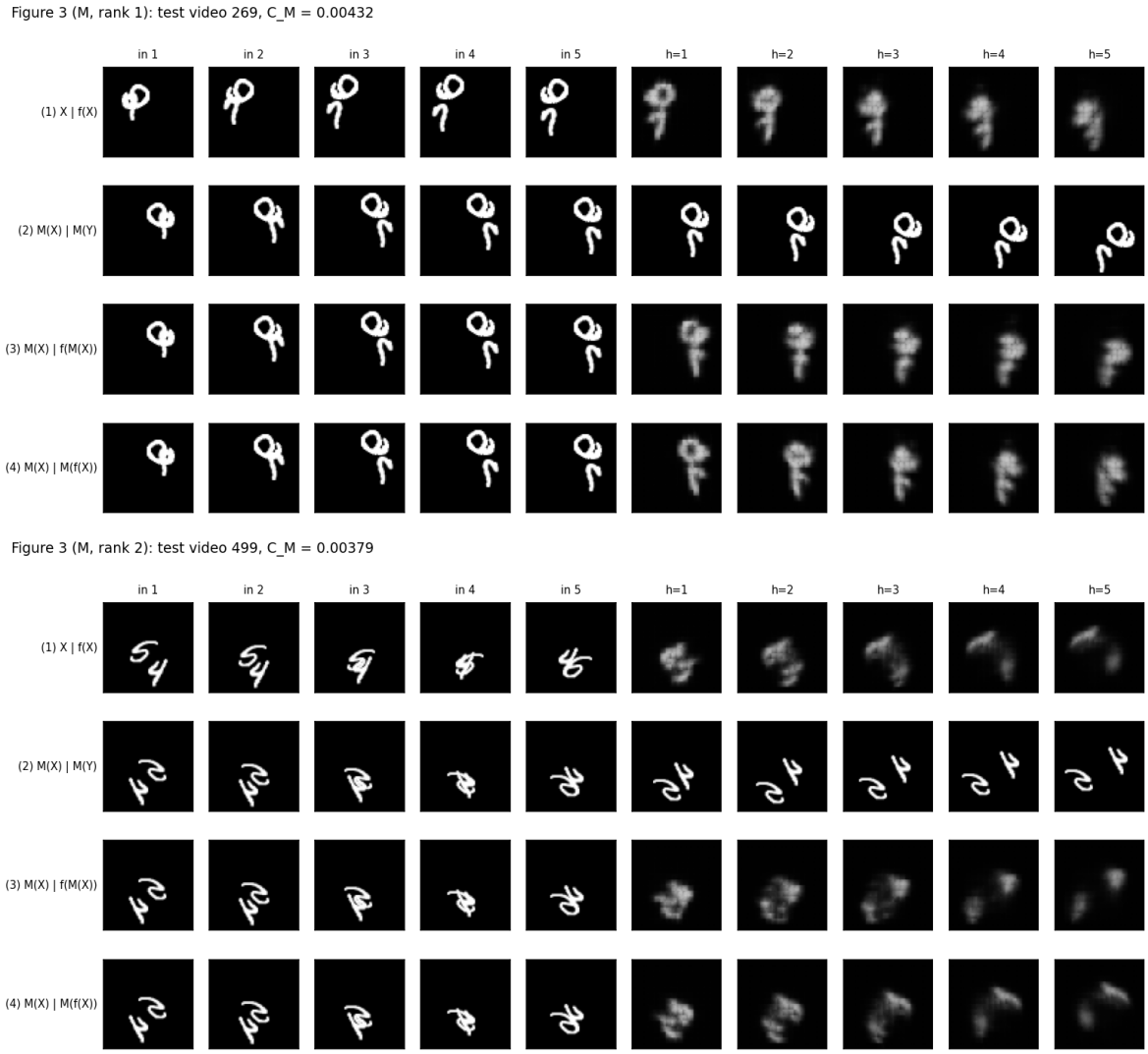


Figure 3 (part 1). The two test videos with the largest per-video C_M for the seed-0 model A (videos 269 and 499). Rows: (1) $X \mid f(X)$, (2) $M(X) \mid M(Y)$, (3) $M(X) \mid f(M(X))$, (4) $M(X) \mid M(f(X))$. GIFs: `gifs/figure3_M_rank1_video269.gif`, `gifs/figure3_M_rank2_video499.gif`.

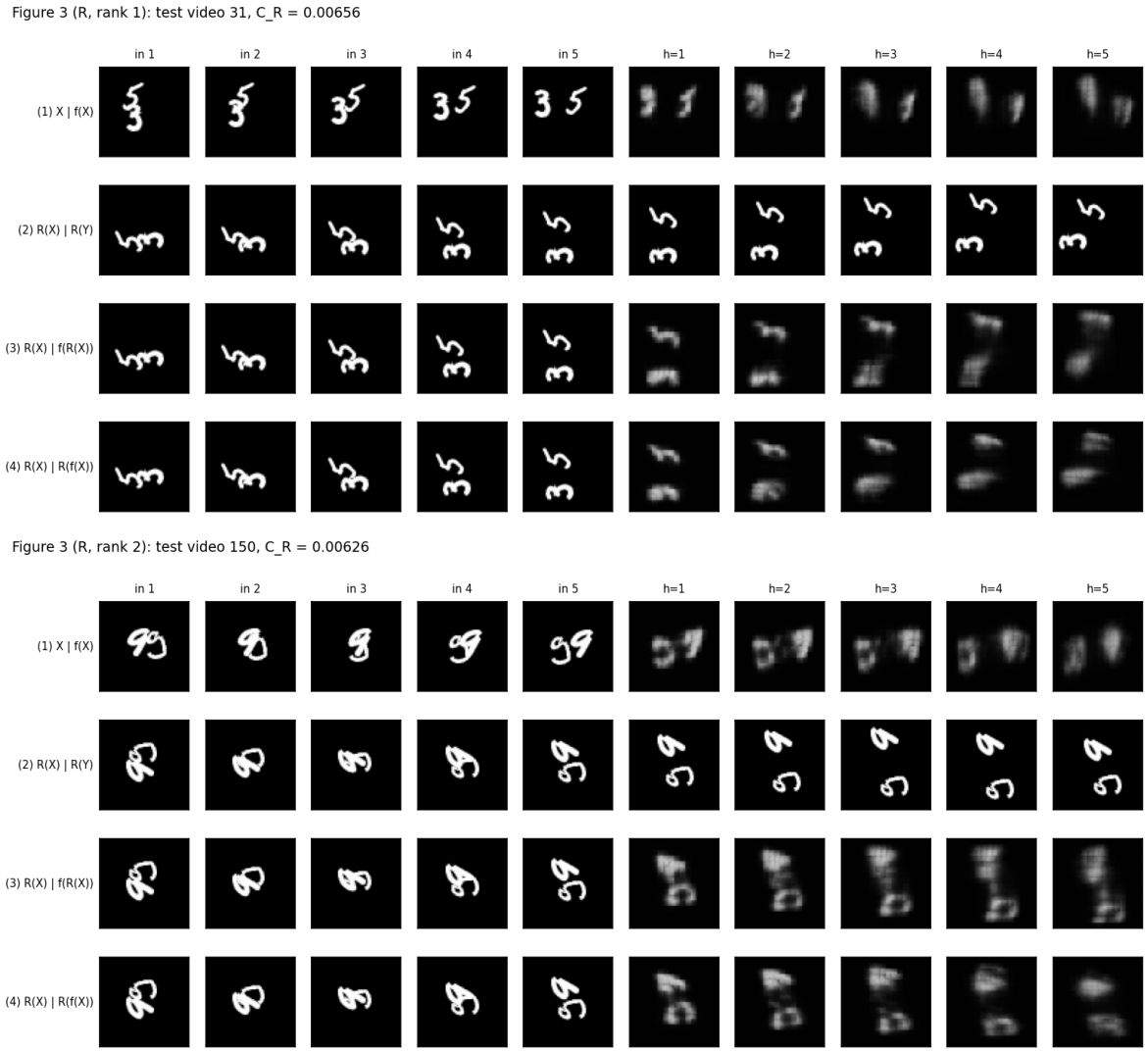


Figure 3 (part 2). The two test videos with the largest per-video C_R for the seed-0 model A (videos 31 and 150). Rows as in part 1 with $T = R$. GIFs: `gifs/figure3_R_rank1_video31.gif`, `gifs/figure3_R_rank2_video150.gif`.

Q2.6 Blank and Copy-last have $C_M = C_R = 0$ exactly (Table 3), yet they are the two worst predictors: their errors are 0.0426 and 0.0641, compared with 0.0222 for model A. They are consistent for a trivial reason: Blank ignores the input completely (a black frame mirrored or rotated is still a black frame), and Copy-last only repeats the last input frame, and copying a frame gives the same result whether one transforms before or after. Neither of them has learned anything about motion. So C_T on its own only tells us whether a model treats a transformed video in the same way as the original; it does not tell us whether the predictions are correct. Read alone, C_T would even rank Blank ($C_M = 0$) above model A ($C_M = 0.00163$), which is clearly wrong; in the same way, a model that outputs a uniform grey frame would be perfectly consistent and useless. This is why E_T must always be reported next to C_T: E_T compares the prediction on the transformed video with the true transformed future $T(Y)$, so it measures correctness. A good model needs both a small C_T (consistent) and an E_T close to its normal error E (correct). Only the reverse direction comes for free: since T only rearranges pixels, $C_T \leq 2(E_T + E)$, so a model that is accurate on both the original and the transformed videos must also be fairly consistent, but a consistent model need not be accurate. (Per pixel, with $a = f(T(X)) - T(Y)$ and $b = T(Y) - T(f(X))$: $(a + b)^2 \leq 2a^2 + 2b^2$; a^2 averages to E_T, and since T only moves pixels, b^2 averages to E.)

Q2.7 In test video 150 under rotation (Figure 3, R rank 2, $C_R = 0.00626$), rows 3 and 4 disagree about the upper digit, the rotated 9. In the true future (row 2), this digit stays near the top of the frame and drifts only slightly to the right. When model A predicts on the original video and the prediction is rotated afterwards (row 4), the digit stays near the top as a wide, blurry blob, which agrees with the truth on where the digit is, although its shape is lost. When model A predicts directly on the rotated video (row 3), its account of the motion changes: from $h = 3$ on, the digit's brightness spreads downwards into a tall vertical streak that reaches towards the lower digit, so the digit blurs into something unrecognisable and the model behaves as if it might be moving down, which it is not. The lower digit moves to the right in a similar way in rows 3 and 4, so the disagreement comes almost entirely from the upper digit. In the first input frames the two 9s overlap, which may explain why the model is unsure which piece belongs to which digit; the other three videos in Figure 3 also start with overlapping digits.

Q2.8 Yes, rotation is harder than mirroring for model A, although both effects are small. Under mirroring the error rises from $E = 0.0222$ to $E_M = 0.0228$ (+3%), and under rotation to $E_R = 0.0240$ (+8%), so rotation's extra error (0.0018) is about three times mirroring's (0.0006). The consistency gap is also larger, $C_R = 0.00288$ against $C_M = 0.00163$, and both seeds show the same ordering, so this is not seed noise. The difference does not come from the motion: in Q2.1 every direction already occurs in training, and both transformations only move directions between familiar bins while keeping the speed the same, so motion under rotation is no less familiar than under mirroring. Using the Figure 1(a) bins, about 13% of the direction distribution moves to a different bin under the mirror and about 16% under rotation, a similar change that cannot explain an extra error about three times larger. What differs is the appearance (Q2.2): a mirror keeps vertical strokes vertical, and nearly symmetric digits such as 0, 1 and 8 hardly change, whereas a 90° rotation turns every digit on its side, making it wider than tall, unlike any training digit. Figure 3 supports this: in videos 31 and 150 the true rotated digits are wide, yet when model A runs on the rotated video (row 3) it draws some of them as tall or tilted streaks, the shape of upright digits, while when it runs on the original video and its output is rotated (row 4), the blobs stay wide like the truth. Model A's motion rule therefore seems partly tied to what upright digits look like, which is why the transformation that changes appearance more costs more. In Q2.5 I predicted the same result for both; the results show they are close, but rotation is measurably harder.

Model	$E^{(2)}$ **1×**	$E^{(2)}$ **2×**	$E^{(2)}$ **3×**	$E^{(5)}$ **1×**	$E^{(5)}$ **2×**
A	0.0189 (0.0191, 0.0186)	0.0442 (0.0435, 0.0449)	0.0511 (0.0509, 0.0514)	0.0222 (0.0225, 0.0220)	0.0484 (0.0476, 0.0493)
Copy-last	0.0548	0.0661	0.0703	0.0641	0.0719

Table 4. Faster motion ($t = 1$). Across speeds only $E^{(2)}$ is compared; at 3× only two targets exist (frames 16, 19). Model A: mean (seed 0, seed 1).

Figure 5. Test video 21 at 1x, 2x, 3x (seed-0 model A): five inputs, first two predictions, first two true frames.

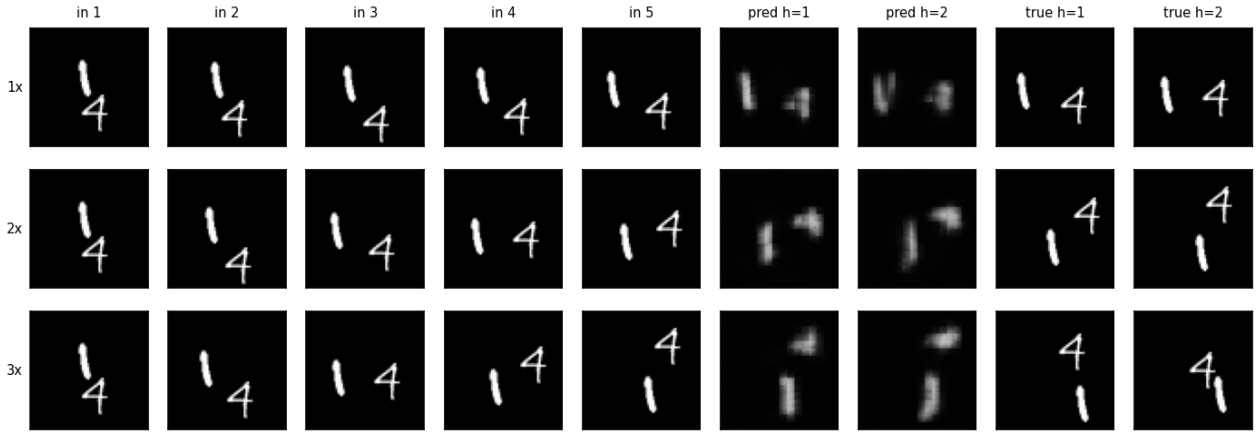


Figure 5. Test video 21 at 1×, 2× and 3× (seed-0 model A): five inputs, first two predictions, first two true frames. The digits do not overlap in the displayed true frames.

Q2.9 Model A's predictions fall apart already at 2×. $E^{(2)}$ rises from 0.0189 at 1× to 0.0442 at 2× (2.3 times higher) and 0.0511 at 3× (Table 4). Blank's error is about 0.0426 at every horizon (Table 2), so from 2× on model A is no better than predicting black frames, and its $E^{(5)}$ at 2× (0.0484) is worse than Blank; it still beats Copy-last (0.0661 at 2×).

This does *not* agree with my reach estimate. At 2× the digits move 6.64 px per step, inside the 8 px reach, so by reach alone model A should cope at 2× and only fail at 3× (9.96 px). The estimate missed two things. First, reach is only an upper limit: the most the architecture *could* move information in one step, not what it actually learned. Model A was trained only at 1×, where digits move about 3.3 px per step, less than one grid cell, so it never needed to move information further than about one cell per step and never learned to use its full reach. Second, the 8 px path uses the outermost weights of the 3×3 kernels in both layers; in practice information near the edge of a kernel's window is carried only weakly, so the useful reach is smaller than the theoretical one.

Figure 5 (test video 21) shows what the failure looks like. At 1× the predicted blobs are blurry but sit where the true digits are. At 2× and 3× the predicted digits *stall*: at $h = 1$ and $h = 2$ they stay close to where they were in the last input frame, while the true digits have moved much further (the 1 moves right; at 3× the 4 bounces and comes back down, but the predicted 4 stays at the top). They also *blur*: the 1 becomes a long vertical smear and the 4 a shapeless blob. No digit vanishes or is duplicated. Stalling suggests the model does not pick up the large velocity: it cannot link a digit's old and new positions when they are far apart, so it falls back on the slow motion it learned in training, or none at all. Blur suggests it is unsure where the digit is going and spreads it over the possible positions, which costs less under squared error than a sharp wrong guess. A vanishing digit would mean the model had lost track of it completely, and a duplicated digit would mean it treated the old and new positions as two different objects; neither happens here, so the model still knows a digit is there, but not how fast it moves.

Q2.10 No. Model A was trained only at 1× and has a fixed reach of 8 px, so on fast videos both

causes are present at once, and both predict a failure; its results alone cannot tell them apart. The failure at 2×, which is inside the reach, hints that unfamiliar motion matters, but it does not prove it, because the useful reach may be smaller than the theoretical 8 px (Q2.9). To separate the two causes, change one thing at a time, as in a controlled experiment. First, train the same architecture on mixed speeds (k drawn uniformly from {1, 2, 3}): fast motion is then no longer unfamiliar, while the reach stays at 8 px. If this model works at 2× and 3×, the failure was caused by unfamiliar motion; if it works at 2× but still fails at 3×, where digits move 9.96 px per step, beyond the 8 px reach, the remaining failure is architectural. Second, to confirm this, train a network with a larger reach (e.g. bigger kernels, dilated convolutions or a bigger patch) on the same mixed speeds: if it fixes 3× where the same-reach model fails, the cause was reach, because more training data cannot carry information further than the architecture allows.

2.4 What can data fix?

Q2.11 *(written before training models B–D; reproduced unedited)*

I predict that model B, trained with random mirrors and rotations, will fix the rotation (and mirror) failure, because it sees transformed digits during training. I predict that model C, trained on mixed speeds, will fix the failure at 2×. I predict that the failure at 3× will be fixed by both model C and model D.

Model D (design). Identical to model A except that the 3×3 convolution of the second ConvLSTM layer is *dilated by 3* (padding 3). Code: `self.cell2.conv = nn.Conv2d(128, 256, kernel_size=3, padding=3, dilation=3)`.

Model	**from u_τ**	**from $h^{(1)}_{\tau-1}$**	**from $h^{(2)}_{\tau-1}$**	**reach (longest)**	**Params**
A	2 cells = 8 px	2 cells = 8 px	1 cell = 4 px	**8 px**	480,784
D	4 cells = 16 px	4 cells = 16 px	3 cells = 12 px	**16 px**	480,784

Table S1. One-step reach *measured* with a gradient check (one step of the untrained network; every input cell with a non-zero gradient for one output cell). For D, every horizontal and vertical offset from −4 to +4 cells is reachable (no holes).

Model	$E^{(5)}$ **1×**	C_M	E_M	C_R	E_R
A	0.0222 (0.0225, 0.0220)	0.00163 (0.00155, 0.00171)	0.0228 (0.0231, 0.0226)	0.00288 (0.00254, 0.00321)	0.0240 (0.0240, 0.0239)
B	0.0229 (0.0233, 0.0224)	0.00111 (0.00106, 0.00115)	0.0229 (0.0234, 0.0224)	0.00134 (0.00141, 0.00128)	0.0228 (0.0232, 0.0223)
C	0.0267 (0.0272, 0.0263)	0.00135 (0.00117, 0.00154)	0.0269 (0.0273, 0.0265)	0.00258 (0.00220, 0.00295)	0.0279 (0.0282, 0.0276)
D	0.0285 (0.0291, 0.0280)	7.74×10^{-4} (7.11×10^{-4}, 8.38×10^{-4})	0.0286 (0.0291, 0.0281)	0.00171 (0.00142, 0.00199)	0.0293 (0.0297, 0.0289)

Table 5 (part a). Models A–D at 1× (test set, $t = 1$): accuracy and consistency. Entries: mean (seed 0, seed 1).

Model	$E^{(2)}$ 1×	$E^{(2)}$ 2×	$E^{(2)}$ 3×	Parameters	Train time per epoch (s)	Inference time (s)
A	0.0189 (0.0191, 0.0186)	0.0442 (0.0435, 0.0449)	0.0511 (0.0509, 0.0514)	480,784	11.0 (10.9, 11.2)	0.386 (0.387, 0.386)
B	0.0192 (0.0197, 0.0187)	0.0431 (0.0433, 0.0429)	0.0490 (0.0496, 0.0483)	480,784	10.8 (10.8, 10.9)	0.388 (0.387, 0.388)
C	0.0232 (0.0237, 0.0227)	0.0288 (0.0292, 0.0285)	0.0333 (0.0335, 0.0331)	480,784	10.5 (10.5, 10.5)	0.389 (0.388, 0.390)
D	0.0259 (0.0264, 0.0254)	0.0296 (0.0300, 0.0292)	0.0332 (0.0335, 0.0328)	480,784	14.8 (14.8, 14.8)	0.549 (0.548, 0.549)

Table 5 (part b). Models A–D: speed and cost (T4 GPU). Training time per epoch from the training logs; inference time = predicting all 1,000 original test videos at batch size 32 under `eval()` and `no_grad()`, measured in one session for all models.

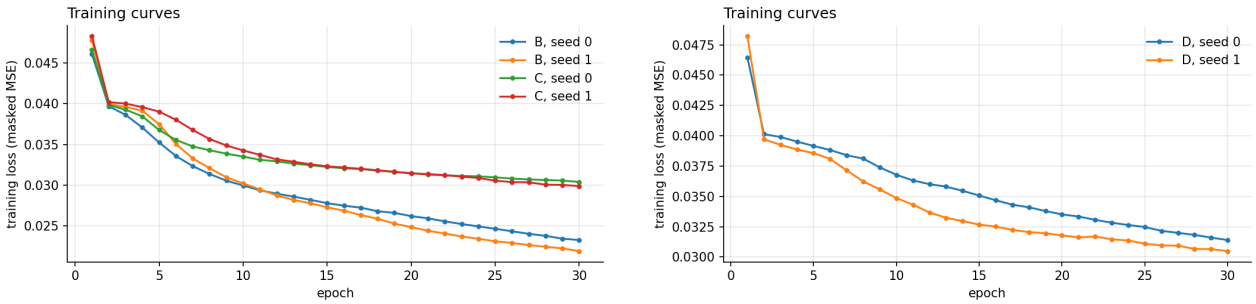


Figure S2. Training loss of models B, C (left) and D (right). Models C and D are trained on mixed speeds, so their training loss is higher than B's; D is still improving at epoch 30. (Supplementary.)

Q2.12 Model D is model A with a single change: the 3×3 convolution of the second ConvLSTM layer is dilated by 3, with padding 3 so the grid stays 16×16. Everything else is identical: patch size 4 (`pixel_unshuffle`/`pixel_shuffle`), two ConvLSTM layers with 64 hidden channels, the 1×1 output convolution, the 9-step rollout without teacher forcing, and the training protocol (mixed speeds, like model C). A dilated 3×3 kernel uses the same 9 weights but places them 3 cells apart, so it costs no extra parameters: D has exactly 480,784 parameters, the same as A and C and well under the 721,176 limit. I chose this design so that D differs from C *only in reach*, not in size: if D beats C, the reason cannot be extra parameters.

Reach, counted as in Q2.3: one grid cell is 4 px, and only the 3×3 convolutions move information (the gates, unshuffle/shuffle and the 1×1 head move nothing). In layer 1 the ordinary 3×3 convolution moves information at most 1 cell; in layer 2 the dilated convolution moves it at most 3 cells. The path from u_τ therefore moves $1 + 3 = 4$ cells, the path from $h^{(1)}_{\tau-1}$ also $1 + 3 = 4$ cells, and the path from $h^{(2)}_{\tau-1}$ 3 cells. The longest path is 4 cells = **16 px** along rows and columns (about 22.6 px diagonally), strictly larger than model A's 8 px and above the 9.96 px per step of 3× motion. Because layer 1 moves information by $-1, 0, +1$ cells and layer 2 by $-3, 0, +3$, together they reach every offset from -4 to $+4$ cells, so there are no holes (unlike dilation 2 in both layers, which would reach only even offsets). A gradient check (Table S1) confirms this: one step reaches 8 px for model A and 16 px for model D, with every offset from -4 to $+4$ cells reachable.

Q2.13 Each label is supported by a comparison in Table 5 in which only one thing changes.

- **Rotation (and the small mirror gap): appearance.** Model B differs from A only by symmetry augmentation, i.e. by showing it mirrored and rotated digits during training. This alone reduces the extra error under rotation from +0.0018 (A: $E_R = 0.0240$ vs. $E = 0.0222$) to about zero (B: $E_R = 0.0228$ vs. $E = 0.0229$); the small mirror gap also disappears (+0.0006 → 0.0000), and the consistency gaps drop (C_R 0.00288 → 0.00134, C_M 0.00163 → 0.00111). The

motion was already familiar (Q2.1), so what B added was new digit appearances. B leaves the speed failures unchanged ($E^{(2)}$ at 2×: 0.0431 vs. 0.0442), which confirms that the two problems are separate.

- **2×: unfamiliar motion.** Model C has the same architecture and the same 8 px reach as A; only its training speeds differ. This alone lowers $E^{(2)}$ at 2× from 0.0442 to 0.0288 (−35%). The digits move 6.64 px per step, inside the reach, so the network could always connect the positions; it had simply never been trained on this motion.
- **3×: mainly unfamiliar motion, not reach.** The comparison A→C again gives the large gain ($E^{(2)}$ at 3×: 0.0511 → 0.0333, −35%). The comparison C→D changes only the architecture, doubling the reach from 8 to 16 px with the same parameters and data, and gives no further gain (0.0333 vs. 0.0332). So in these results reach was not what limited the model at 3×, even though 9.96 px per step is above 8 px along an axis. Possible reasons (my interpretation): digit strokes are wide, so a digit's old and new positions still partly overlap within 8 px; the diagonal reach is up to about 11.3 px; and constant velocity can be estimated across several steps. Two caveats: C's error at 3× is still clearly higher than at 1× (0.0333 vs. 0.0232), so 3× is improved, not solved; and D's training curve was still falling at epoch 30 (Figure S2), so a longer-trained D might show some benefit.

If model C had still done badly at 3× while model D did well, that would have shown that the 3× failure was architectural reach: C had seen 3× motion in training and still failed, and only the larger reach fixed it. More training data cannot get past that limit, because training only changes the numbers inside each kernel, while the set of cells that can affect an output in one step is fixed by the kernel size, dilation, patch size and number of layers. If a digit moves further than that in one step, there is no path in the network connecting its old position (in the state) with its new position; the gradient of the output with respect to those far-away inputs is exactly zero, so no amount of data can teach the network to use them.

Q2.14 Yes. Every change cost some accuracy on the original 1× clips ($E^{(5)}$ at 1×, Table 5): B slightly (0.0229 vs. 0.0222, +3%), C clearly (0.0267, +20%) and D the most (0.0285, +28%).

- **B: worth it.** For +3% at 1× it removes the rotation gap ($E_R - E$ from +0.0018 to −0.0001) and halves C_R.
- **C: worth it whenever the model has to handle faster motion**, which is the point of this assignment; it would not be worth it for a forecaster that only ever sees normal-speed videos. The cost at 1× (+0.0045 in $E^{(5)}$) buys much larger gains at 2× ($E^{(2)}$ −0.0154) and 3× (−0.0178); at those speeds A was no better than predicting black frames, while C is well below that level. The 1× cost is expected (interpretation): only a third of C's training windows are at 1×, and the same weights must now cover three speeds, so the model hedges more.
- **D: not worth it here.** Compared with C, which has the same training data, it is worse at 1× (0.0285 vs. 0.0267), no better at 2× (0.0296 vs. 0.0288) or 3× (0.0332 vs. 0.0333), and about 40% slower to train (14.8 vs. 10.5 s per epoch) and to run (0.549 vs. 0.389 s). A likely reason for its 1× loss (interpretation): layer 2's dilated kernel only links cells that are 0 or 3 apart, so small 1× movements (less than one cell per step) can be handled only by layer 1, and D's predictions look blockier (Figure 6); D was also still improving at epoch 30.

2.5 The final model

Why the symmetrised forecaster is exactly consistent. Take any trained f and any $h \in G$. Then $\bar{f}(h(X)) = \frac{1}{8}\sum_{g\in G} g^{-1} f(ghX)$. Substitute $g' = gh$. Because G is a group, $g \mapsto gh$ maps G one-to-one onto itself, so g' runs over all eight elements exactly once, and $g^{-1} = hg'^{-1}$. Since h only

rearranges pixels, it is linear and can be taken out of the average:

$$\bar{f}(h(X)) = \tfrac{1}{8} \sum_{g' \in G} h\, g'^{-1} f(g' X) = h\big(\bar{f}(X)\big).$$

With $h = M$ and $h = R$ this gives $\bar{f}(T(X)) = T(\bar{f}(X))$, so $C_M = C_R = 0$ exactly for any f. (With a subset of G that is not closed, the re-indexed sum would run over different elements and the equality would fail.) In floating point the eight terms are added in a different order, so the measured values are about 10^{-17} (Table 6).

Choice of final model. I use the handout's default: model D wrapped as a symmetrised forecaster. I did not train D+B: symmetrisation already makes the model exactly consistent, and Table 5 shows no benefit of D's extra reach that would justify further training of D variants.

Model	$E^{(5)}$ **1**×	C_M	E_M	C_R	E_R
symmetrised B	0.0223 (0.0227, 0.0218)	7.63×10^{-17} (8.24×10^{-17}, 7.03×10^{-17})	0.0223 (0.0227, 0.0218)	6.17×10^{-17} (6.68×10^{-17}, 5.67×10^{-17})	0.0223 (0.0227, 0.0218)
symmetrised D (final model)	0.0284 (0.0289, 0.0278)	6.54×10^{-17} (6.53×10^{-17}, 6.55×10^{-17})	0.0284 (0.0289, 0.0278)	5.15×10^{-17} (5.09×10^{-17}, 5.21×10^{-17})	0.0284 (0.0289, 0.0278)

Table 6 (part a). Symmetrised models at 1× (test set, $t = 1$). C_M, C_R in scientific notation. Entries: mean (seed 0, seed 1).

Model	$E^{(2)}$ **1**×	$E^{(2)}$ **2**×	$E^{(2)}$ **3**×	**Parameters**	**Train time per epoch (s)**	**Inference time (s)**
symmetrised B	0.0187 (0.0192, 0.0183)	0.0425 (0.0427, 0.0423)	0.0482 (0.0488, 0.0476)	480,784	10.8 (10.8, 10.9)	3.24 (3.22, 3.26)
symmetrised D (final model)	0.0258 (0.0263, 0.0253)	0.0296 (0.0300, 0.0291)	0.0331 (0.0335, 0.0327)	480,784	14.8 (14.8, 14.8)	4.73 (4.69, 4.78)

Table 6 (part b). Symmetrised models: speed and cost (T4 GPU). Training time is that of the underlying learned model (symmetrisation needs no training); inference time includes all eight transformations, forward passes, inverse transformations and the averaging.

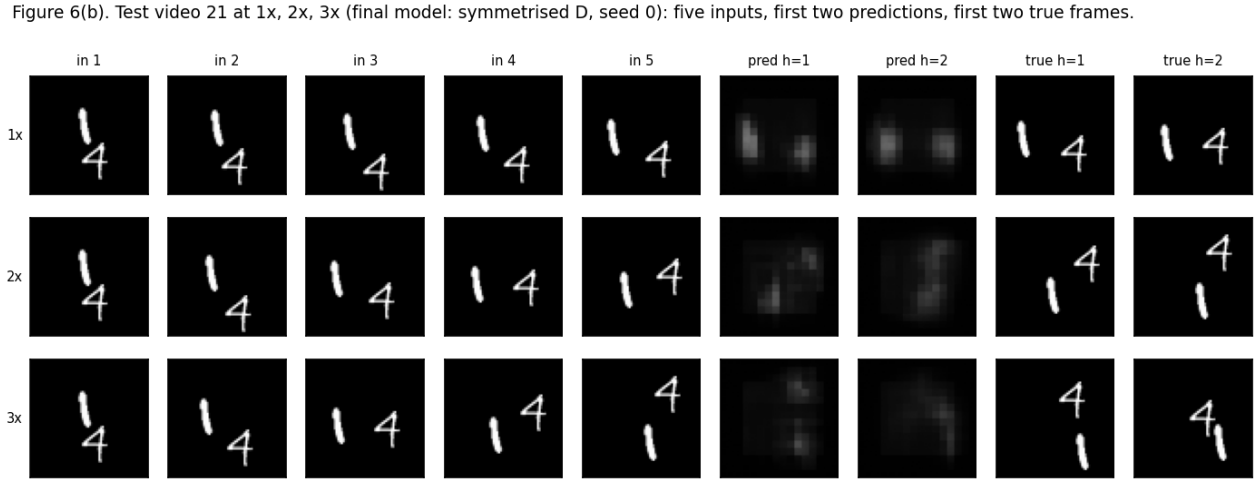


Figure 6(b). Figure 5 repeated with the final model (symmetrised D, seed 0) on test video 21.

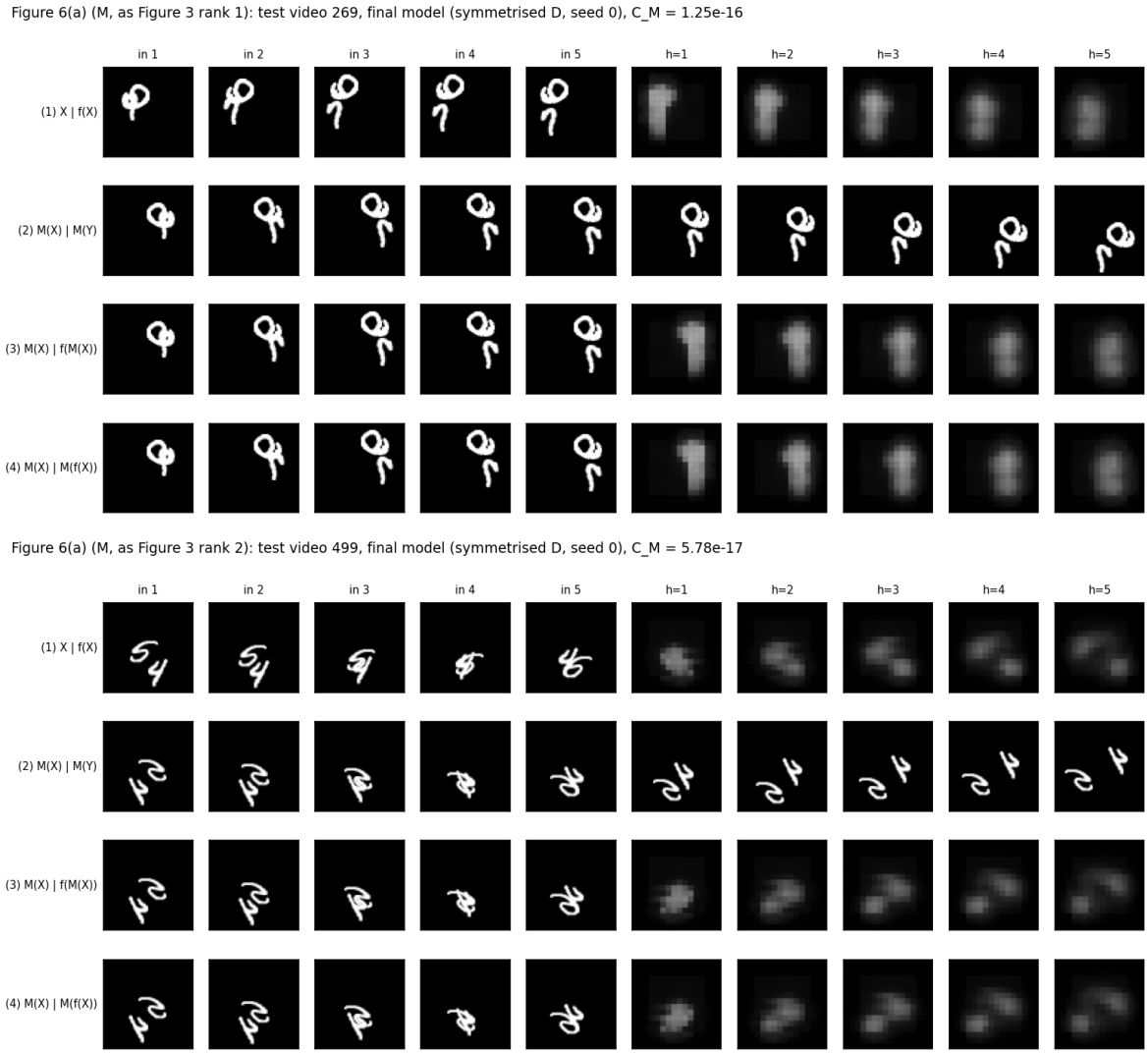


Figure 6(a), part 1. Figure 3 repeated with the final model (symmetrised D, seed 0) on the same videos 269 and 499 ($T = M$); rows as in Figure 3. GIFs: `gifs/figure6a_M_rank1_video269.gif`, `gifs/figure6a_M_rank2_video499.gif`.

Figure 6(a) (R, as Figure 3 rank 1): test video 31, final model (symmetrised D, seed 0), C_R = 5.87e-17

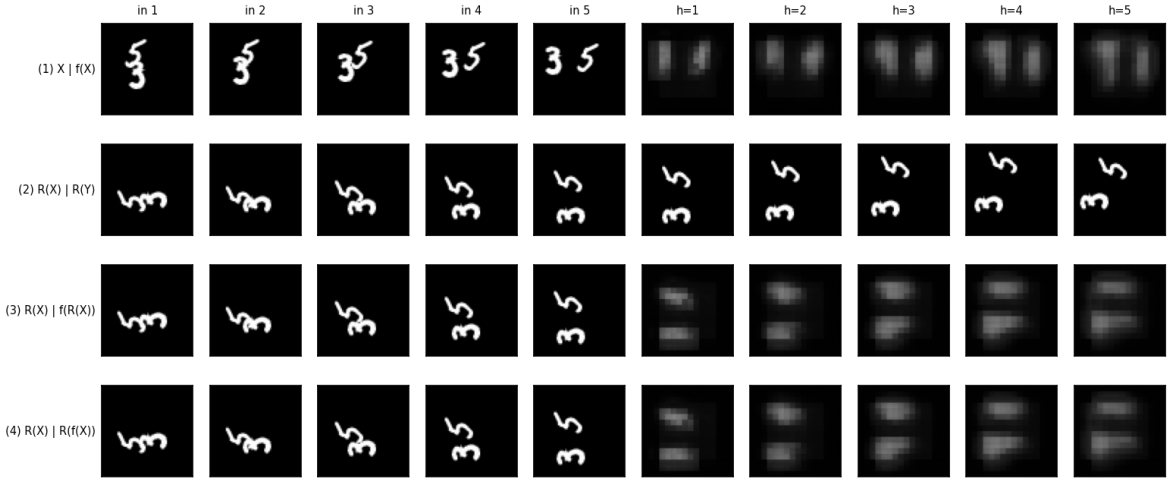


Figure 6(a) (R, as Figure 3 rank 2): test video 150, final model (symmetrised D, seed 0), C_R = 9.60e-17

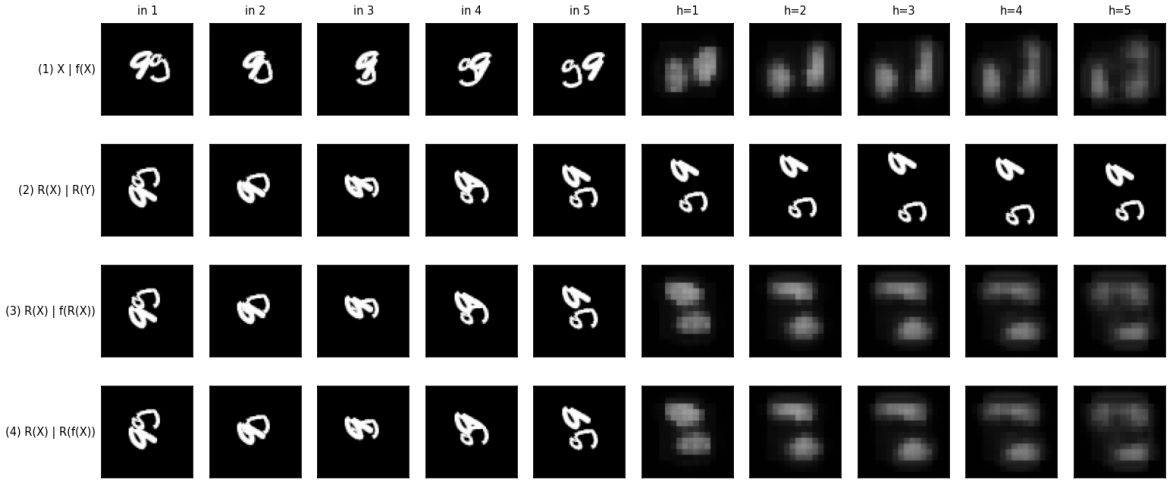


Figure 6(a), part 2. Videos 31 and 150 ($T = R$) with the final model. GIFs: `gifs/figure6a_R_rank1_video31.gif`, `gifs/figure6a_R_rank2_video150.gif`.

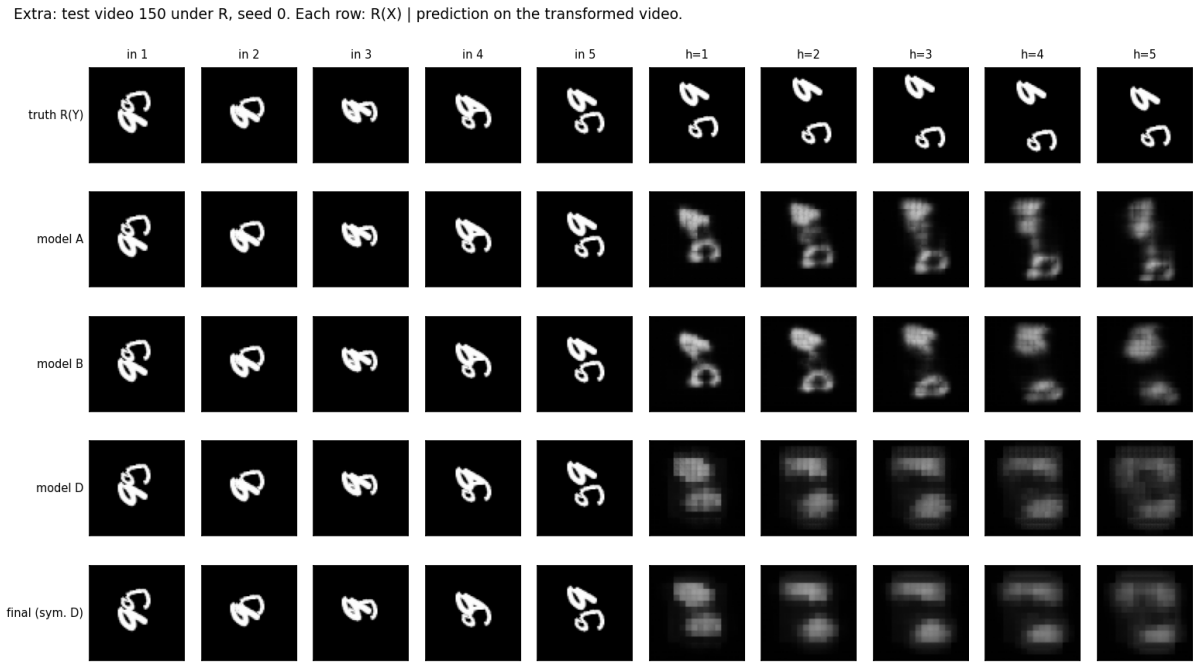


Figure S3. Test video 150 under rotation (seed 0): truth, then the prediction on the rotated video by models A, B, D and the final model. (Supplementary, for Q2.16.)

Q2.15 **Consistency.** Learned symmetry (model B) reduces the gaps but does not remove them: from A to B, C_M goes from 0.00163 to 0.00111 and C_R from 0.00288 to 0.00134. Built-in symmetry removes them: symmetrised B and symmetrised D have C_M and C_R of about 10^{-17}, zero up to floating-point error. Built-in symmetry is exact because of the group argument above, which holds for any f whatever its weights. Learned symmetry is only encouraged: augmentation adds transformed examples to the loss, but with finite training, finite capacity and an optimiser that only lowers the average loss, nothing forces $f(T(X))$ and $T(f(X))$ to be identical for every input.

Accuracy. Because the symmetrised models are exactly equivariant and mirror/rotation only rearrange pixels, their errors on transformed clips equal their errors on original clips exactly: $E_M = E_R = E = 0.0223$ for symmetrised B and 0.0284 for symmetrised D. Model B gets close but not exact (E 0.0229, E_M 0.0229, E_R 0.0228). On original clips the symmetrised models are as good as or slightly better than the learned ones: symmetrised B 0.0223 against B's 0.0229 (equal to A's 0.0222, while being far better than A on rotated clips, where A's E_R was 0.0240); symmetrised D 0.0284 against D's 0.0285.

Inference cost. Symmetrisation needs no training but runs the model eight times per prediction: 3.24 s against 0.388 s for B (8.4×) and 4.73 s against 0.549 s for D (8.6×). Symmetry augmentation costs nothing at inference and nothing extra in training (B 10.8 s per epoch vs. A 11.0 s).

Blur. Averaging blurs the prediction when the eight predictions disagree. I would expect this when the base model works well only for one orientation (like model A with upright digits), so that seven of the eight predictions are worse and place the digits in slightly different places, or when the data are not truly symmetric (e.g. gravity, or digits whose orientation matters), so that forcing symmetry removes useful information. Here it did not hurt: the motion rule and the square box are symmetric under all eight transformations, model B was trained on all eight so its eight predictions already agree closely, and averaging acted like a small eight-member ensemble: $E^{(5)}$ at 1× went down for B (0.0229 → 0.0223) and stayed the same for D (0.0285 → 0.0284). The final model's predictions in Figure 6 are blurry, but mostly because model D's own predictions are blurry and blocky (Figure S3), not because of the averaging.

Q2.16 Only partly. In Q2.7, model A's prediction on the rotated video 150 disagreed with its

rotated normal prediction (per-video $C_R = 0.00626$): the upper digit smeared downward into a tall streak, as if moving towards the lower digit. With the final model (symmetrised D), rows 3 and 4 of Figure 6(a) are identical (per-video $C_R \approx 10^{-16}$), and the downward streak is gone: both digits stay as blobs in roughly the right places, the upper one near the top and the lower one drifting right. So the model's account of the motion is now consistent and no longer goes the wrong way. But the prediction is not more accurate: the error on this rotated video is 0.0419 for the final model against 0.0298 for model A, because model D's predictions on this video are much blurrier and blockier, and the digits' shapes are lost. Figure S3 shows which change fixed which part:

- The wrong-way smear came from appearance: symmetry augmentation (B) already removes most of it, keeping the upper digit near the top and lowering both the error (0.0274) and the consistency gap ($C_R = 0.00211$).
- The remaining disagreement between rows 3 and 4 is removed exactly by symmetrisation, which forces C_R to zero for any model; symmetrised B gives the most accurate version of this video (0.0262).
- The speed changes did not help here and in fact made this slow ($1\times$) video worse (C 0.0393, D 0.0429): this is the $1\times$ cost found in Q2.14, larger here because this video, which starts with the two 9s overlapping, is one of the hardest for D (35th worst of 1,000).

Q2.17 Partly. The final model learned a motion rule that works in every orientation and at all three trained speeds, but only coarsely: digits become blurry blobs. Three pieces of evidence would convince a skeptic. First, controlled comparisons: each change fixed one failure. Symmetry augmentation (B) removed the rotation gap, mixed-speed training (C) cut the $2\times$ and $3\times$ errors by about 35%, and doubling the reach (D) added nothing. Second, exact consistency with low error: $C_M = C_R \approx 10^{-17}$ and $E_M = E_R = E = 0.0284$, well below Blank (0.0426); consistency alone would not be enough, as Blank shows. Third, at normal speed (Figure 6) the predicted blobs follow the true paths instead of stalling. One test it has not passed: speeds outside training. It was only tested at $1\times$–$3\times$; a $4\times$ test needs longer or newly generated videos, so we cannot claim it learned speed in general.

Appendix: Use of generative AI

Tool. Claude (Anthropic), used through the Claude desktop app between 13 and 21 October 2026, as a tutor and assistant. As required by the course AI policy, this appendix lists my prompts, what the AI produced, and the edits I made. A per-question log is also in the repository (`ai_log.md`). All training and evaluation runs were executed by me on Kaggle; the AI never ran my real experiments (it only tested notebooks on synthetic data).

Part	My prompt (short)	AI output	My work / edits
Understanding the task	"explain me this assignment completely"; "a fast and smart way of doing this assignment"	Explanation of both questions; a day-by-day plan.	Followed the plan; no text used in the report.
Q2 Step 1 (data, transforms, windows, Table 1)	"how to start"; "lets do it on kaggle"; "elaborate cell 3"	Kaggle notebook scaffold (loading, splits, Table 1 checker) with explanations; the four functions were left as TODOs.	I wrote `mirror`, `rotate`, `make_window`, `apply_g_inverse`; the AI pointed out my mistakes (e.g. `k=a` in `rot90`, the target mask) and I fixed them.
Q2 Step 2 (Figure 1)	"explain the whole concept, then I will implement"	Notebook scaffold (Farneback call, plotting); explanation of optical flow.	I wrote `keep_mask` and `direction_deg`.
ConvLSTM background	"explain this ConvLSTM paper with diagrams"; "draw the FC-LSTM / ConvLSTM diagram"	A 17-page LaTeX study guide and diagrams (Shi et al., 2015).	For my own study only; not used in the report.
Q2 Step 3 (model A)	"explain everything in detail, after that I will implement"; later "give me the codes in cells"	Scaffold with automatic checks; after my attempts, a completed reference version of the cells.	I wrote `ConvLSTMCell.forward`, `ModelA.step/forward` (with corrections); the final cells match the reference.
Q2 Step 4 (training)	"what to do while GPU training"	Training notebook (protocol, seeds, timing, logs, models A–D).	I wrote `masked_mse`; I ran all training on Kaggle.
Q2 Steps 5–6 (evaluation, Tables 2–6, Figures 2–6)	"give me the notebook"	Evaluation notebooks with automatic checks.	I wrote the C_T/E_T lines and the line of `symmetrise` (the AI corrected a wrong function name); I ran both notebooks.
Model D	"model D"	Explanation of reach levers; table of candidate designs with parameter counts and measured reach; the 3-line `ModelD` class after my choice.	I computed D1's reach and the over-budget layer myself and chose design D2.

Part	My prompt (short)	AI output	My work / edits
Q2.1–Q2.4, Q2.6–Q2.10, Q2.12–Q2.17	e.g. "help me write it"; "write a final polished version"; "give me final ans"	Taught each answer through check questions (typed or multiple choice); then wrote the final wording from my answers and my results, and corrected my errors (e.g. Q2.1 median vs. uniform line; Q2.9 reach agreement; Q2.10 "bigger kernels"; Q2.14 verdict on C; Q2.15 which symmetry is exact). For Q2.8 and Q2.16 it computed extra numbers from my result files.	I answered the checks, made the judgements, and checked each answer against my tables and figures.
Q2.5 and Q2.11 (predictions)	"write it for me"	Declined to choose the predictions; offered reasons drawn from my own Q2.1–Q2.3 results as options, then turned my choices into sentences.	All predictions and reasons were chosen by me before the experiments; reproduced unedited.
Q1 (all six headings)	"tutorial question 1"; "I want to complete the assignment today"	Read the course's Q1 tutorial slides with me; after I made the main design decisions, wrote the full Q1 text, assumptions, data table, equations and the TikZ diagram, following the tutorial's skeleton.	I chose: 6-h window, directed graph with distances, median + 10/90% quantiles. I reviewed the text and can explain each part.
Report and repository	"LaTeX → PDF"	Assembled this LaTeX report (tables converted from my result files, my figures inserted) and drafted `README.md` and `ai_log.md`.	Reviewed and submitted.